

Development of a Python SDK for the Blueframe Embedded Platform

Project Requirements and Specifications



03-SEL-BFSDK

Caleb Hansen

Genevieve Kochel

Darron Li

Lucas Phillips

TABLE OF CONTENTS

I. Introduction.....	3
II. System Requirements Specification.....	3
II.1. Use Cases.....	3
Use Case 1: Engineer Connects to SignalHub and Subscribes to Device Signals.....	3
Use Case 2: New Engineer Onboards to Blueframe Development.....	4
Use Case 3: Engineer Connects to SignalHub and Publishes Data to a Channel.....	5
II.2. Functional Requirements.....	6
II.2.1. SignalHub Integration:.....	6
II.2.1. Authentication and Authorization.....	6
III.2.3 API Abstraction Layer.....	6
II.3. Non-Functional Requirements.....	7
II.4 Traceability Matrix.....	7
II.5 Standards.....	7
III. System Evolution.....	7
IV. Glossary.....	8
V. References.....	8

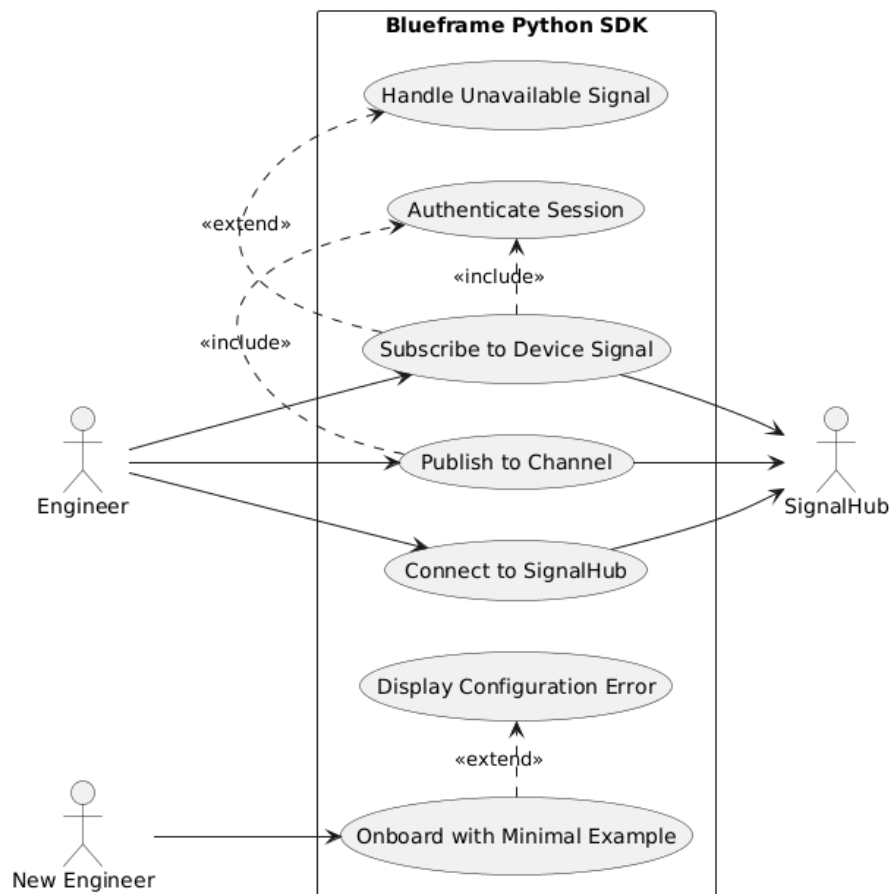
I. Introduction

For many decades, we have relied on the electrical power grid to provide critical, and at times life-saving, infrastructure. The security and reliability of the electrical grid, therefore, is paramount. SEL Blueframe is an operating system and platform designed with this in mind. It is built from a hardened, stripped-down version of the Linux kernel that runs entirely containerized applications, along with many other security features. Blueframe is often run at edge locations; embedded computing devices such as the SEL 3350 are installed at power substations and run the Blueframe operating system.

The goal is extremely high reliability and availability - we should spend minimal time installing patches and updates, and should minimize the chance of system crashes or malicious attacks. As a result of this specialized design, application development for Blueframe is restricted to those with specialized understanding of system APIs and internals. and thus essentially limited to SEL engineers. Our Python SDK project aims to address this gap and create a clean abstraction over the Blueframe system API to make development much more accessible.

II. System Requirements Specification

II.1. Use Cases



Use Case 1: Engineer Connects to SignalHub and Subscribes to Device Signals

Precondition	- The engineer has installed the Blueframe Python SDK in a containerized application. - The engineer has access credentials for SignalHub.
Postcondition	- The engineer successfully subscribes to a device signal and receives real-time data
Basic Path	1. The engineer imports the SignalHub module. 2. The engineer initializes a client with authentication handling. 3. The SDK establishes a secure session with SignalHub automatically. 4. The engineer subscribes to a specific device signal using a high-level method. 5. The engineer receives real-time device data for analysis.
Alternative Path	- If the device signal is unavailable, the system notifies the engineer and offers a retry option.
Related Requirements	[FR1] Secure Connection Establishment [FR3] Subscribe to Channel [FR6] Pythonic Interface

Use Case 2: New Engineer Onboards to Blueframe Development

Precondition	- The newly hired engineer has access
--------------	---

	to the Blueframe Python SDK documentation. - A test device is available for initial connection.
Postcondition	- The engineer successfully builds a minimal Blueframe application.
Basic Path	1. The engineer reads the Blueframe Python SDK documentation. 2. The engineer copies a minimal working example. 3. The engineer modifies only configuration parameters to connect to the test device. 4. The engineer runs the application and verifies connectivity and basic functionality.
Alternative Path	- If configuration parameters are incorrect, the system displays clear error messages.
Related Requirements	[FR6] Pythonic Interface [FR7] General System Calls

Use Case 3: Engineer Connects to SignalHub and Publishes Data to a Channel

Precondition	- The engineer has installed the Blueframe Python SDK in a containerized application. - The engineer has access credentials for SignalHub.
Postcondition	- The engineer successfully publishes device data to a SignalHub channel.

Basic Path	1. The engineer imports the SignalHub module. 2. The engineer initializes a client with authentication handling. 3. The SDK establishes a secure session with SignalHub automatically. 4. The engineer publishes device data to a channel or signal within SignalHub.
Alternative Path	- If authentication fails, the system prompts the engineer to re-enter credentials. - If the channel does not exist or is unavailable, the system notifies the engineer and allows selection of a different channel.
Related Requirements	[FR1] Secure Connection Establishment [FR2] Publish to Channel [FR6] Pythonic Interface

II.2. Functional Requirements

II.2.1. SignalHub Integration:

FR1 - Secure Connection Establishment

The system must establish an authenticated session with SignalHub using provided credentials and configuration parameters.

Source: Joe Stanley

Priority: Priority Level 0

FR2 - Publish to Channel

The system must allow users to publish structured messages to a specified SignalHub channel using a single high-level method call.

Source: SEL Electrical and Computer Engineers

Priority: Priority Level 0

FR3 - Subscribe to Channel

The system must allow users to subscribe to one or more SignalHub channels and receive messages through a high-level callback interface.

Source: SEL Engineers

Priority: Priority Level 0

II.2.1. Authentication and Authorization

FR4 - Secure Credential Handling

The system must securely accept and store authentication credentials without exposing them in logs or error messages.

Source: Project security requirements

Priority: Priority Level 0

FR5 - Permission Validation

The system must validate user or application permissions before executing protected operations.

Source: Project security requirements

Priority: Priority Level 0

III.2.3 API Abstraction Layer

FR6 - Pythonic Interface

The system must provide idiomatic Python interfaces that abstract low-level SignalHub APIs.

Source: SEL Engineers

Priority: Priority Level 0

FR7 - General System Calls

The system provides generalized, reliable, and high-level Python interfaces for interacting with low-level Blueframe APIs outside of SignalHub.

Source: SEL Engineers

Priority: Priority Level 2

II.3. Non-Functional Requirements

NFR1 - Security

The system shall be secure by default and ensure that all communication, credential handling, and session management comply with Blueframe's security standards.

NFR2 - Performance

The system shall execute API calls and handle data efficiently, introducing minimal computational or memory overhead compared to direct API usage.

NFR3 - Reliability

The system shall handle transient failures gracefully, ensuring predictable behavior and automatic recovery where appropriate.

NFR4 - Maintainability

The system shall be structured, documented, and organized to support long term maintenance and extension by SEL engineers and future developers.

NFR5 - Usability

The system shall be intuitive and accessible for engineers without a formal computer science background, providing clear abstractions and examples.

II.4 User Stories and Traceability Matrix

II.4.1 User Stories

US1

As an engineer, I want to authenticate and establish a secure session with SignalHub, so that I can interact with device signals.

US2

As an engineer, I want to subscribe to a device signal using a high-level method, so that I can receive real-time data.

US3

As an engineer, I want to be notified and retry when a device signal is unavailable, so that I can maintain continuous data access.

US4

As a new engineer, I want to use a minimal working example and modify only configuration parameters, so that I can quickly build and verify a Blueframe application.

US5

As a new engineer, I want clear error messages when configuration parameters are incorrect, so that I can correct setup issues efficiently.

US6

As an engineer, I want to publish device data to a SignalHub channel using a high-level method, so that other systems can consume the data.

II.4.2 Traceability Matrix

Stakeholder Need	User Story ID	Functional Requirement	Nonfunctional Requirement	Priority
Establish secure connection to SignalHub	US1	FR1 – Secure Connection Establishment	NFR1 – Security	Priority Level 0
Receive real-time device data	US2	FR3 – Subscribe to Channel	NFR3 – Reliability	Priority Level 0

Handle unavailable device signals gracefully	US3	FR3 – Subscribe to Channel	NFR3 – Reliability	Priority Level 0
Rapid onboarding and minimal setup effort	US4	FR6 – Pythonic Interface	NFR5 – Usability	Priority Level 0
Clear feedback on configuration errors	US5	FR6 – Pythonic Interface	NFR5 – Usability	Priority Level 0
Publish device data to SignalHub	US6	FR2 – Publish to Channel	NFR2 – Performance	Priority Level 0

II.5 Standards

Standard	Domain	Description	Applications
IEEE 29148-2018: Systems and Software Requirements Engineering	Requirements Engineering	Defines processes and content for high quality requirements specifications (successor to IEEE 830)	Used to structure Functional Requirements, Non-Functional Requirements, use cases, and traceability matrix
IEEE/ISO/IEC 12207: Software Life Cycle Processes	Software Lifecycle Management	Defines lifecycle stages, processes, documentation, and configuration management practices	Applied to requirements, design, implementation, testing, and maintenance planning
ISO/IEC 27001:2022: Information Security Management Systems	Cybersecurity / Information Security	Specifies requirements for establishing, implementing, and maintaining information security controls	Guides secure credential handling (FR4), authentication (FR1), permission validation (FR5), and secure communication (NFR1)

III. System Evolution

The biggest consideration we need to account for is maintainability. Both the Blueframe OS itself and the software that runs on it frequently changes, and the system is expected to go through multiple rounds of testing, iteration, and refinement. Some of the risk points in our design process are the SignalHub APIs themselves – these are subject to change and update – and reliance on third party libraries to handle logic. In the event of the SignalHub APIs changing, we would have to either adapt our code logic to the new versions of the APIs, or create legacy versions of the APIs that can be called by our code. If a third party library we use changes or becomes deprecated, we would need to either create an in-house replica of the library, or rewrite our code to not use that logic. Due to the highly secure, embedded environment these applications will be run in, we should work to minimize external dependencies as much as reasonably possible.

Some assumptions that we are making about the project are that the Blueframe will always be running a secure and stable Python environment, and that it will be using containerized deployments for applications. Engineers are assumed to have a basic understanding of Python and to have access to the SDK documentation.

IV. Glossary

OS - Operating System, the low-level software on a computer that enables the hardware to run applications.

IED - Intelligent Electronic Device.

Blueframe - Custom hardened operating system developed by SEL.

SDK - Software Development Kit, a set of convenient tools and wrappers for developers to write software (typically targeting a specific platform).

SEL - Schweitzer Engineering Laboratories.

V. References

[1] OpenAPI Initiative, *OpenAPI Specification*, OpenAPI Initiative, 2024. [Online]. Available: <https://www.openapis.org>. Accessed: Feb. 5, 2026.

[2] Swagger (SmartBear Software), *Swagger: API Development Tools*, SmartBear Software, 2024. [Online]. Available: <https://swagger.io>. Accessed: Feb. 6, 2026.

[3] Schweitzer Engineering Laboratories, *Blueframe Platform*, Schweitzer Engineering Laboratories, Inc., Pullman, WA, USA, 2024. [Online]. Available: <https://selinc.com>. Accessed: Feb. 1, 2026.